

Guía técnica de implementación

Sistema de Control de Entradas PAE — cómo funciona por dentro, para el equipo que lo instala, configura y da soporte.

Esta guía es complementaria a la "Guía para operadores" (que explica solo lo que necesita saber quien opera el escáner el día a día). Aquí se documenta el funcionamiento completo del sistema: arquitectura de archivos, almacenamiento de datos, ciclos automáticos, reportes, configuración de correo/nube y empaquetado con PyInstaller — todo lo necesario para instalarlo en una institución nueva, diagnosticar problemas o extenderlo.

Componentes del paquete

Archivo	Rol
entradas_pae26.py	Programa principal: interfaz de escaneo, sincronización de datos, disparo de los ciclos diario/mensual y de reportes.
reporte_quincenal.py	Genera el PDF del reporte quincenal (gráficas con matplotlib + tablas con reportlab).
reporte_mensual_detallado.py	Genera el PDF del reporte mensual por persona (candidatos a ceder cupo de alimentación).
entradas_pae26.spec	Receta de PyInstaller para compilar todo lo anterior en un .exe distribuible.

Los tres .py deben vivir siempre en la misma carpeta — los dos módulos de reportes se importan como módulos locales (`import reporte_quincenal`, `import reporte_mensual_detallado`) y fallan en silencio (con aviso en el log) si no están presentes, sin afectar el resto del programa.

1. Arquitectura de carpetas y archivos

Al primer arranque, el programa crea por sí solo todo lo que le falte dentro de la carpeta donde vive (BASE_DIR — la carpeta del .py, o la del .exe si está compilado):

Ruta	Contenido
datos/	Toda la persistencia: IDs, historiales, base de datos y archivos de configuración.
datos/ids_primaria.xlsx, ids_secundaria.xlsx, ids_docentes.xlsx	Roster editable a mano por los operadores (ver Guía para operadores).
datos/historial_total.xlsx	Una hoja por grado con las marcas de asistencia (X) del mes en curso, más la hoja "Estadísticas".
datos/vhistorial_primaria.xlsx, vhistorial_secundaria.xlsx, vhistorial_docentes.xlsx	Historial plano de las entradas del día en curso (se vacía cada reinicio diario).
datos/asistencia.db	Espejo en SQLite de lo mismo, para que los reportes consulten con SQL en vez de recorrer Excel.
datos/correo_config.txt, directivos_correo.txt, carpeta_reportes.txt	Configuración de envío de reportes (ver sección 6).
datos/ultimo_dia.txt, ultimo_mes.txt, ultimo_reporte.txt, ultimo_reporte_mensual.txt	Punteros de estado que disparan los ciclos automáticos (ver secciones 4 y 5).
datos/app.log, app.lock	Bitácora de eventos/errores y bloqueo simple para detectar otra instancia abierta.
reportes/	Copia local GARANTIZADA de todos los PDF generados (ver sección 6).
BASE_DIR cambia según cómo se ejecute Corriendo el .py directo: BASE_DIR es la carpeta que contiene entradas_pae26.py. Corriendo el .exe compilado con PyInstaller: BASE_DIR es la carpeta que contiene el .exe (no la carpeta temporal donde PyInstaller descomprime en memoria) — así el .exe puede vivir en cualquier ruta y su carpeta "datos" lo acompaña.	

2. Almacenamiento de datos: Excel + SQLite en paralelo

Cada entrada válida se escribe DOS veces, en el mismo momento: una vez en los archivos Excel (para que el personal los pueda abrir y leer directamente) y otra en asistencia.db (para que los reportes puedan usar SQL en vez de recorrer muchas celdas). Ambas escrituras están envueltas en try/except independientes: si SQLite falla, el programa sigue funcionando con Excel con normalidad, y viceversa.

Tablas de asistencia.db

Tabla	Columnas	Se llena en
personas	id, nombre, apellido, grado, categoria	sync_personas_db(), espejo de los archivos ids_*.xlsx
entradas	id (autoincr.), persona_id, fecha, hora	Cada escaneo válido (verify_entry)
estadisticas_diarias	fecha, categoria, porcentaje	Al cierre de cada día (ejecutar_reinicio_diario)

Archivos Excel

- ids_*.xlsx (uno por categoría): columnas ID, Nombre, Apellido; una pestaña por grado/grupo. Es el ÚNICO Excel editable a mano.
- historial_total.xlsx: fila 1 = fechas del mes (formato dd-mm), fila 2 = encabezados, fila 3+ = personas; una X marca la fecha en que esa persona entró. Se reinicia cada mes (ver sección 5).
- vhistorial_*.xlsx: log plano (Fecha, Hora, ID, Nombre, Apellido, Grado) de las entradas del día — se vacía en cada reinicio diario, después de volcar el % del día a la hoja "Estadísticas".

CATEGORIAS (línea ~81 de entradas_pae26.py) es la lista que define cuántas categorías existen y dónde vive cada archivo. Agregar una categoría nueva (por ejemplo "Personal administrativo") es tan simple como agregar un diccionario más a esa lista — el resto del programa (sincronización, historial, reportes) se adapta solo, sin tocar nada más.

3. Ciclo diario

- Al escanear un ID (verify_entry): valida contra personas cargadas en memoria, evita duplicados del mismo día, escribe en vhistorial_*.xlsx + historial_total.xlsx (X en la columna de hoy) + tabla entradas de asistencia.db, y reproduce sonido de error si el ID no es válido.
- Reinicio diario (ejecutar_reinicio_diario): se dispara al detectar que ultimo_dia.txt ya no coincide con la fecha de hoy (al abrir el programa, o a medianoche si se dejó abierto). Calcula el % de asistencia del día que terminó por categoría, lo agrega a la hoja "Estadísticas" de historial_total.xlsx y a estadísticas_diarias en la base de datos, y vacía los vhistorial_*.xlsx para el día nuevo.

app.lock — no es un bloqueo real

Al abrir, el programa avisa en el log si ya existe app.lock (probable señal de que el programa sigue abierto en otra máquina sobre la misma carpeta compartida), pero NO impide abrir una segunda instancia — así nadie se queda fuera por un bloqueo viejo tras un cierre inesperado. Si dos instancias escriben al mismo tiempo sobre el mismo Excel, sí pueden pisarse los cambios; evitar carpeta compartida abierta desde dos equipos a la vez.

4. Ciclo mensual

- sync_historial_total(): en cada arranque, lee los tres ids_*.xlsx, crea las hojas de grado que falten en historial_total.xlsx, agrega personas nuevas y actualiza nombre/apellido de las que ya existían (sin borrar nunca asistencia ya marcada). Si el mismo ID aparece en más de una categoría o archivo, lo anota como advertencia en app.log y se queda con el último registro leído.
- gestionar_limpieza_mensual(): se dispara cuando ultimo_mes.txt ya no coincide con el mes actual — archiva/reinicia las columnas de fecha de historial_total.xlsx para empezar el mes limpio, conservando lo ya generado y guardando en la carpeta /históricas una copia completa del Excel de cada mes.

5. Reportes automáticos

Ambos reportes leen directamente de asistencia.db (no de los Excel) y se disparan por comparación de identificadores de periodo guardados en archivos de texto — nunca por temporizador ni por hora exacta.

Reporte	Periodo	Archivo de control	Identificador
Quincenal	1–15 y 16–fin de cada mes	ultimo_reporte.txt	AAAA-MM-Q1 / AAAA-MM-Q2
Mensual detallado	Mes calendario completo	ultimo_reporte_mensual.txt	AAAA-MM

Lógica de disparo (idéntica en ambos, `gestionar_reporte_quincenal()` / `gestionar_reporte_mensual()`): al arrancar (y también al detectar cambio de día a medianoche si el programa se deja abierto) se calcula el identificador del periodo ACTUAL y se compara contra el guardado en el archivo de control.

- Si son iguales: no se hace nada, seguimos dentro del mismo periodo.
- Si son distintos: se genera el reporte para el periodo GUARDADO (el que acaba de terminar), usando ese rango de fechas contra asistencia.db, y luego se actualiza el archivo de control al identificador actual.

Consecuencia práctica para soporte

El reporte de una quincena/mes NUNCA aparece el mismo día en que cierra ese periodo — aparece la primera vez que se abre el programa YA iniciado el periodo siguiente. Si alguien reporta "no se generó el reporte", lo primero es revisar la fecha real del sistema contra el contenido de `ultimo_reporte.txt` / `ultimo_reporte_mensual.txt` antes de asumir un error.

`MINIMO_DIAS_CON_DATOS` (`reporte_mensual_detallado.py`, default 5) evita generar el reporte mensual si el mes tuvo muy pocos días con datos (mes recién empezado); en ese caso simplemente no se genera y se reintenta el mes siguiente.

`UMBRAL_CANDIDATO` (`reporte_mensual_detallado.py`, default 0.50) define el % de asistencia mensual por debajo del cual una persona se marca como "Candidato" a ceder su cupo de alimentación en el reporte mensual. Es la única cifra pensada para ajustarse según el criterio de cada institución.

6. Respaldo local y distribución (nube + correo)

Cada PDF (quincenal o mensual) se genera SIEMPRE primero dentro de la carpeta local "reportes", junto al programa — ese respaldo nunca se salta, sin importar la configuración de nube o correo. A partir de ahí, `distribuir_reporte()` intenta copiarlo también a una carpeta de nube si está configurada, y `enviar_reporte_por_correo()` intenta enviarlo por correo — ambos pasos están en try/except: si fallan (permisos, carpeta desconectada, credenciales vencidas), el reporte de todas formas ya quedó guardado localmente, y solo se anota el problema en `app.log`.

carpeta_reportes.txt — copia adicional en la nube

Vacío o solo con comentarios por default (ninguna copia adicional). Si se escribe ahí la ruta de una carpeta sincronizada (Google Drive, OneDrive, etc.), cada reporte se copia también ahí — la app de sincronización se encarga sola de subirlo.

correo_config.txt + directivos_correo.txt — envío automático

Mientras falte cualquiera de los 4 valores obligatorios (servidor, puerto, remitente, contraseña_app) o no haya destinatarios en `directivos_correo.txt`, el envío simplemente no ocurre — sin errores, sin afectar la generación del reporte.

Clave	Gmail (plan gratis)	Proton Mail (Mail Plus o superior)
servidor	smtp.gmail.com	127.0.0.1 (Proton Mail Bridge debe estar abierto en el equipo)
puerto	465	El que muestre Bridge (usualmente 1025, pero puede variar)
contraseña_app	"Contraseña de aplicación" generada en la cuenta de Google — no la contraseña normal	Contraseña generada por Bridge — no la contraseña de la cuenta Proton
seguridad	ssl (default)	starttls

directivos_correo.txt: un correo por línea; todos reciben copia de AMBOS reportes (el asunto y cuerpo del correo distinguen "quincenal" de "mensual" automáticamente vía el parámetro tipo de enviar_reporte_por_correo).

7. Empaquetado con PyInstaller

entradas_pae26.spec ya incluye reporte_quincenal y reporte_mensual_detallado como hiddenimports de respaldo (por si PyInstaller no los detecta solos al ser imports locales). Para compilar, los cinco archivos deben estar juntos en la misma carpeta:

- entradas_pae26.py, reporte_quincenal.py, reporte_mensual_detallado.py, entradas_pae26.spec
- recursos/logo.ico (opcional, ícono del ejecutable)

Comandos de compilación

```
pip install pyinstaller pandas openpyxl playsound3 matplotlib reportlab
```

```
python -m PyInstaller entradas_pae26.spec
```

El .exe resultante queda en dist/. La carpeta "datos" y "reportes" se crean solas la primera vez que se ejecuta, junto al .exe — no hace falta copiarlas a mano.

8. Diagnóstico y logs

app.log (dentro de "datos") registra, con fecha y hora, cada evento relevante: creación de archivos, sincronización de IDs, IDs duplicados, limpieza mensual, generación/envío de reportes y cualquier excepción no esperada. Es el primer lugar a revisar ante cualquier reporte de una discrepancia, antes de tocar cualquier archivo del sistema.

Síntoma	Dónde revisar primero
El contador no coincide con las personas cargadas	IDs duplicados o filas vacías en ids_*.xlsx — ver app.log
No llega el correo con el reporte	correo_config.txt completo y directivos_correo.txt con al menos un correo; ver app.log para el error SMTP exacto
No aparece el reporte todavía	Comparar la fecha real contra ultimo_reporte.txt / ultimo_reporte_mensual.txt (sección 5) — casi siempre es que el periodo no ha cerrado, no un error
El reporte no se genera nunca	Confirmar que matplotlib, reportlab y pandas estén instalados, y que reporte_quincenal.py / reporte_mensual_detallado.py estén junto a entradas_pae26.py (o incluidos en el .exe)

9. Checklist de implementación en una institución nueva

- Copiar entradas_pae26.py, reporte_quincenal.py, reporte_mensual_detallado.py (o el .exe compilado) a la carpeta definitiva.
- Instalar dependencias: pandas, openpyxl, playsound3, matplotlib, reportlab (omitir si se usa el .exe ya compilado).
- Abrir el programa una vez para que se generen solas las carpetas "datos" y "reportes" y las plantillas de configuración.
- Cargar los roster reales en ids_primaria.xlsx, ids_secundaria.xlsx, ids_docentes.xlsx (ver Guía para operadores).
- Configurar correo_config.txt y directivos_correo.txt si se quiere envío automático.
- Configurar carpeta_reportes.txt si se quiere copia adicional en una carpeta de nube.
- Abrir el programa de nuevo y confirmar que el conteo por categoría refleja el número real de personas cargadas.
- Entregar al operador la "Guía para operadores" y hacer un respaldo inicial de la carpeta "datos".